

SENTINEL AI / Optimization Module

This is the AI/Optimization backend for SENTINEL (Geoagentic Framework to Support Emergency Movement). This module operates completely decoupled from the FastAPI backend and SUMO simulation, exposing a pure data-in/data-out functional facade.

1. Environment Setup

This module is designed to run in a standalone Python environment. **Tested and fully supported on Python 3.12.**

(Note: Python 3.14 requires building pydantic-core from source due to PyO3 compatibility limits and is not officially supported).

```
# Create and activate a clean virtual environment
python3.12 -m venv .venv
source .venv/bin/activate

# Install dependencies (no workaround env vars needed for 3.12)
pip install -r requirements-ai.txt
```

Important Environment Variables

- **SENTINEL_MODEL_DIR:** Must be an **absolute path** to the directory containing `sentinel_models.pkl`. If this is missing, invalid, or relative, the system safely triggers a "baseline_fallback" source in predictions with the reason "missing or invalid SENTINEL_MODEL_DIR".
- **OMP_NUM_THREADS:** Must be set to "1" for stable performance with XGBoost inside the simulator/optimizer loop.

2. Integration / Merging with the Backend & SUMO

To integrate this module into the larger system, the backend team should observe the following constraints:

1. **The Facade** (`sentinel_ai/api.py`): This is the **only** entry point the backend should ever call. All API functions are purely functional and accept Pydantic state objects.
2. **Contract Responses:** Every response model (defined in `sentinel_ai/contracts.py`) includes `contract_version`, `model_version`, `source` (e.g. `ml` or `baseline_fallback`), `confidence`, `reasons`, and `evidence`.
3. **Data Requirements** (`sentinel_ai/data/loader.py`): Currently, `load_dataset("sumo")` raises a `NotImplementedError`. It must be explicitly implemented by the backend team to load real SUMO features matching the exact column spec.
4. **Unit Standards:** ETA is always in **minutes**, distances in **meters**, speed in **km/h**, and timestamps are **UTC floats**.

3. The Public API Endpoints

All entry points accept complete state objects (e.g., `WorldState`). Below are the primary functions exposed in `sentinel_ai/api.py`:

- `predict_eta(req: PredictEtaRequest)`: Predicts p10, p50, and p90 travel times for a given route using the XGBoost ML models.

- `predict_route_risk(req: PredictRouteRiskRequest)`: Returns the probability of route failure (excessive congestion or deadlock) based on hazard proximity and closures.
- `generate_and_score_routes(req: GenerateRoutesRequest)`: Generates diverse K-shortest path routes (primary, backup, contingency) to an incident and scores them.
- `compute_resilience(req: ComputeResilienceRequest)`: Evaluates the redundancy and safety of generated routes against existing hazards and closures.
- `rank_destinations(req: RankDestinationsRequest)`: Filters hospitals by capacity/capability requirements and ranks them by the expected incident-to-hospital travel time.
- `optimize_mission(req: OptimizeMissionRequest)`: Uses OR-Tools CP-SAT to select the best responder units for an incident while minimizing total end-to-end mission time.
- `simulate_counterfactual(req: SimulateCounterfactualRequest)`: Answers "What-If" queries by temporarily modifying the world state (e.g., closing an edge) and re-running the CP-SAT optimizer.
- `replan(req: ReplanRequest)`: A wrapper for `simulate_counterfactual` that also measures controller latency during active missions.

4. Development Commands

- `make test`: Run all unit tests (uses fixed seeds and operates deterministically).
- `make train`: Train the XGBoost models on simulated data and save to `models/`.
- `make demo`: Run the deterministic flagship simulation loop step-by-step.
- `make experiments`: Run large-scale experiments comparing baselines to SENTINEL and generate performance plots/CSV data.

5. Known Limitations (Important for Backend Team)

1. **Prediction Fallback Artifacts**: The ML predictive models (`use_prediction=True`) systematically over-estimate synthetic grid ETAs due to scalar mismatches between synthetic network distances and realistic speed bounds (e.g. producing 95.5 min ETAs on a 10x10 grid).
2. **Default Routing**: `use_prediction` defaults to `False` in the simulator. This allows the system to use the Baseline B exact-simulator shortest path routing loop while retaining full access to mission-level CP-SAT unit assignment.
3. **ML Controller Evaluation**: In stress-test scenarios (seeds 250-299), the ML controller underperformed compared to Baseline B on hazard and mixed scenarios.
4. **Risk & What-If Non-Monotonicity**: Risk models highly overestimate failure near hazards (often hitting 100%), and the XGBoost models are inherently noisy/non-monotonic in what-if scenarios (e.g., closing a road might theoretically decrease predicted ETA slightly).
5. **CP-SAT Mission Overhead**: `optimize_mission` evaluates the *entire route chain* (`Unit -> Incident -> Hospital`), while Baseline B strictly evaluates `Unit -> Incident`. This structural mismatch causes Sentinel's reported mission times to appear artificially higher during direct closed-loop comparisons.
6. **Infinite Loops in Simulator**: If an ambulance is fully surrounded by impassable hazards/closures, a progress guard (every 5 stalled iterations) forces a fallback mechanism to prevent thread hanging.